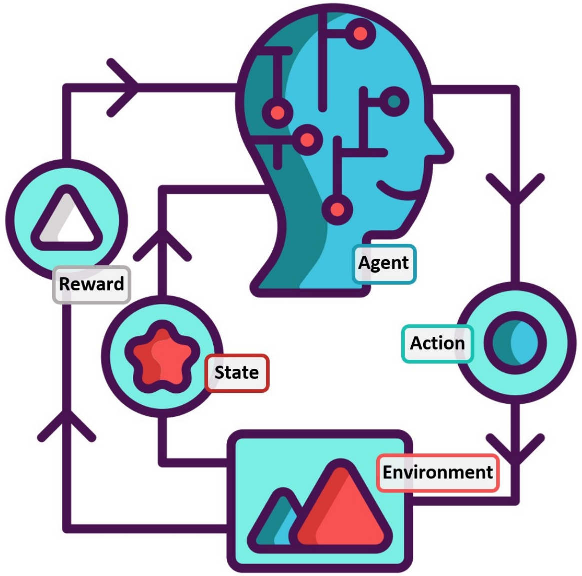
ARTIFICIAL INTELLIGENCE, MACHINE LEARNING AND CONTROL RESEARCH GROUP

Reinforcement Learning
2025/2026

RL2
Reinforcement Learning Research Lab @ UniPD

Lecture #11
Rewind: The Story So Far

Gian Antonio Susto



Announcements before starting

- Partial exam rooms have been announced
- Bring a calculator!

- We did not get the audio recording of the lab also this time... apologies! We are uploading last year lab recording

Question #01

To quickly ask you for clarification about the Bellman equations in the context of MDPs:
I haven't understood what the difference is between the **Bellman equation** and the **Bellman expectation equation**, because from the lecture slides it looks like the goal is to derive (combined) formulas for the **state-value** and **action-value** functions.

The Bellman Equations

Richard E. Bellman

Bellman equations are our tool to 'solve' Markov Reward Processes (MRPs) and MDPs thanks to their recursive nature:

MRP	Bellman equation: for finding value functions	Linear: we can use it for 'small' MRPs. We need to resort to iterative approaches for 'large' MRPs
MDP	Bellman expectation equation: for finding value functions and action-value functions	Linear: we can use it for small MDPs. We need to resort to iterative approaches for 'large' MDPs
MDP	Bellman optimality equation: for finding optimal value functions and optimal action-value functions	Non-linear: we need iterative approaches even for small MDPs.

Question(s) #02

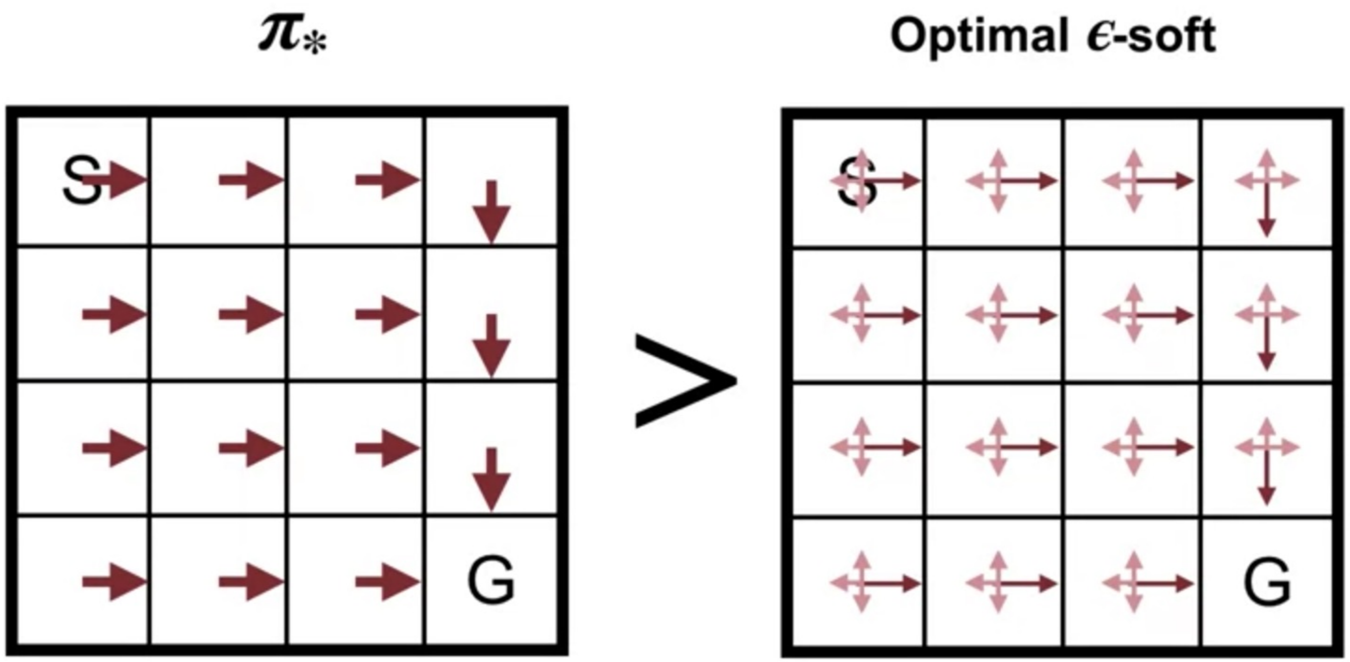
When we studied the **On-policy Monte Carlo control algorithm (for ε-soft policies)**, we said that the resulting policy is not optimal because it is non-deterministic. In this case, one could derive a deterministic policy from the output policy. Would this be a valid solution? In doing so, the algorithm would become very similar to **SARSA**, but using the **Monte Carlo** approach instead of **TD learning** — is that correct?

In the material from Lecture 7, it is stated that *"in MC methods we need to collect data for each (state, action) pair to ensure reaching optimality; however, this is not guaranteed, for example, with deterministic policies."*

I would like to better understand in what sense using a deterministic policy can prevent us from reaching optimality. Is it correct to say that, by always choosing the same action for each state, we do not collect enough data to correctly estimate all the Q(s, a) values?
And therefore, that exploration (for example through stochastic policies or exploring starts) is needed to ensure that every (state, action) pair is visited?

Control: ε-soft policies are not optimal!

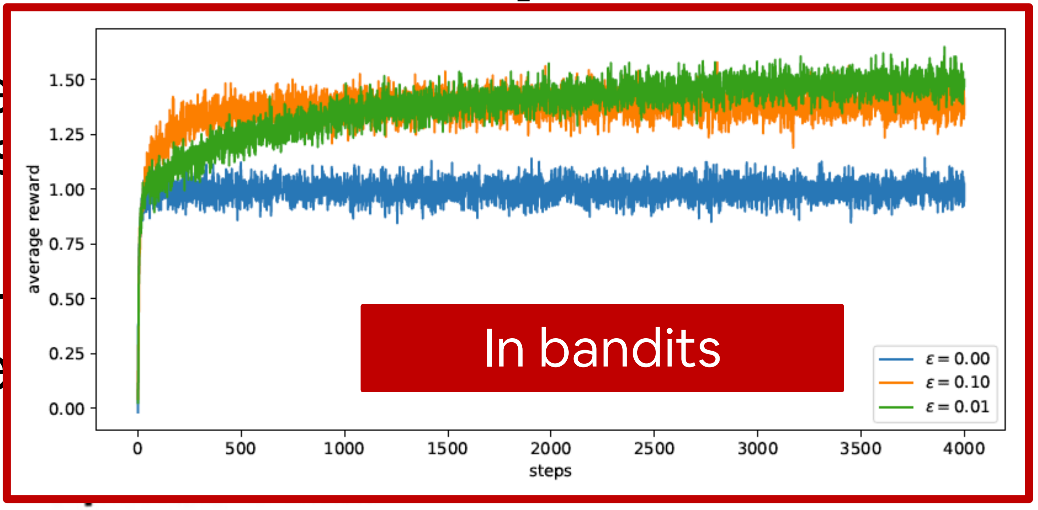
- It can be shown (policy improvement theorem) that the previous algorithm allow improvement (optional, see the book)
- ε-soft policies are not optimal!
- However, they may work quite well and may represent a feasible option when ES cannot be applied (which is true in many cases!)

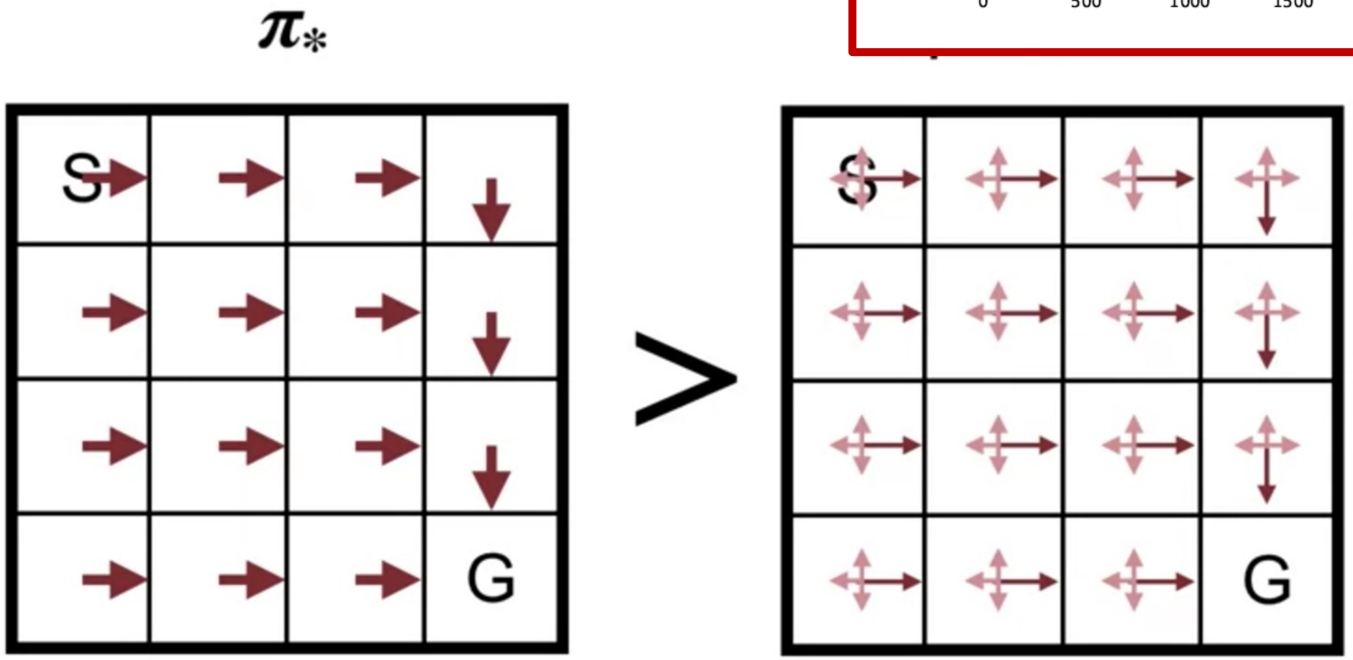


A. White, M. White 'Sample-based Learning Methods'

Control: ε-soft policies are not optimal!

- It can be shown (policy improvement the algorithm allow improvement (optional, s
- ε-soft policies are not optimal!
- However, they may work quite well and r when ES cannot be applied (which is true





A. White, M. White 'Sample-based Learning Methods'

Question #03

In class, we said that **Expected SARSA** is *off-policy*, but in the textbook it says that, by default, if the policy π is used in the summation of the TD target, then Expected SARSA is *on-policy*. The book then explains that we can make it *off-policy* by using a different policy in the target than the one being followed — for example, if we use the **greedy policy** in the target, the summation reduces to a **max**, and Expected SARSA effectively becomes **Q-learning**.

Control: TD control and Bellman equations

$$q_\pi(s,a) = \sum_{s',r} p(s',r|s,a)\left(r + \gamma \sum_{a'} \pi(a'|s')q_\pi(s',a')\right)$$

$$q_*(s,a) = \sum_{s',r} p(s',r|s,a)\left(r + \gamma \max_{a'} q_*(s',a')\right)$$

Sarsa

$$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$$

On-policy

Expected Sarsa

$$R_{t+1} + \gamma \sum_{a'} \pi(a'|S_{t+1})Q(S_{t+1}, a')$$

Off-policy

Q-learning

$$R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a')$$

Off-policy

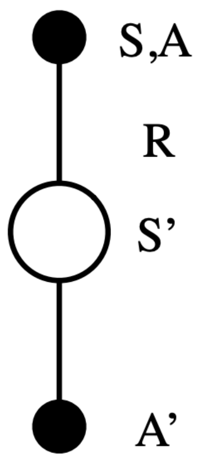
You look ahead and average over potential next actions and then you update Q(s,a)!

A. White, M. White 'Sample-based Learning Methods'

Control: Q-learning vs. SARSA

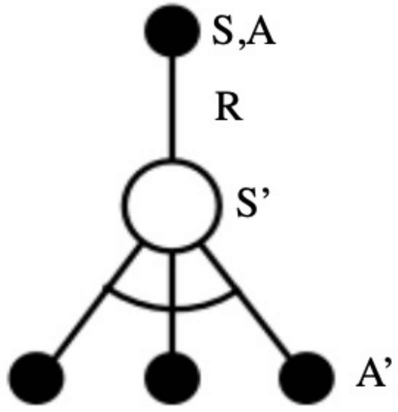
SARSA

Take action A, observe R, S'
Choose A' from S' using policy derived from Q (e.g., ε-greedy)
$Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma Q(S',A') - Q(S,A)\right]$
$S \leftarrow S'$; $A \leftarrow A'$;



Q-learning

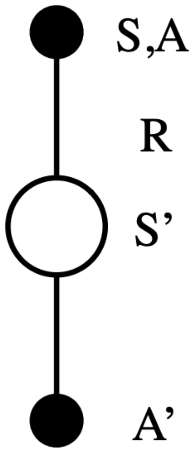
Choose A from S using policy derived from Q (e.g., ε-greedy)
Take action A, observe R, S'
$Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma \max_a Q(S',a) - Q(S,A)\right]$
$S \leftarrow S'$



Control: Q-learning vs. SARSA

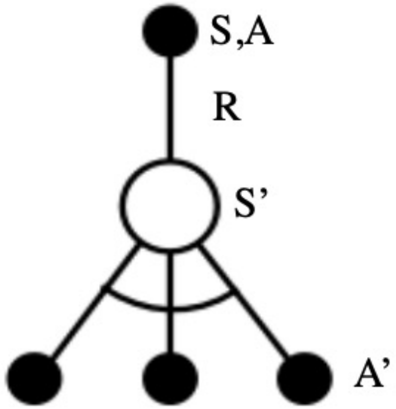
SARSA

Take action A, observe R, S'
Choose A' from S' using policy derived from Q (e.g., ε-greedy)
$Q(S, A) \leftarrow Q(S, A) + \alpha\left[R + \gamma Q(S', A') - Q(S, A)\right]$
$S \leftarrow S'$; $A \leftarrow A'$;



Q-learning

Choose A from S using policy derived from Q (e.g., ε-greedy)
Take action A, observe R, S'
$Q(S, A) \leftarrow Q(S, A) + \alpha\left[R + \gamma \max_a Q(S', a) - Q(S, A)\right]$
$S \leftarrow S'$



The ORDER makes the difference between on-policy and off-policy!

Algorithm 15: Expected Sarsa

Input: policy π, positive integer $num\_episodes$, small positive fraction α

Output: value function Q ($\approx q_\pi$ if $num\_episodes$ is large enough)

Initialize Q arbitrarily (e.g., $Q(s,a) = 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$, and $Q(terminal\text{-}state, \cdot) = 0$)

for $i \leftarrow 1$ ***to*** $num\_episodes$ **do**

    Observe S_0

    $t \leftarrow 0$

    repeat

        Choose action A_t using policy derived from Q (e.g., ϵ-greedy)

        Take action A_t and observe R_{t+1}, S_{t+1}

        $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \sum_a \pi(a|S_{t+1})Q(S_{t+1}, a) - Q(S_t, A_t))$

        $t \leftarrow t + 1$

    until S_t *is terminal*;

end

return Q

You look ahead and average over potential next actions and then you update Q(s,a)

Algorithm 15: Expected Sarsa

Input: policy π, positive integer $num\_episodes$, small positive fraction α
Output: value function Q ($\approx q_\pi$ if $num\_episodes$ is large enough)
Initialize Q arbitrarily (e.g., $Q(s,a) = 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$, and $Q(terminal\text{-}state, \cdot) = 0$)
for $i \leftarrow 1$ ***to*** $num\_episodes$ **do**

    Observe S_0
    $t \leftarrow 0$
    repeat
        Choose action A_t using policy derived from Q (e.g., ϵ-greedy)
        Take action A_t and observe R_{t+1}, S_{t+1}
        $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t))$
        $t \leftarrow t + 1$
    until S_t *is terminal*;
end
return Q

You look ahead and average over potential next actions and then you update Q(s,a)

You can make an on-policy version of Expected SARSA if you change the order of things....

Question(s) #05

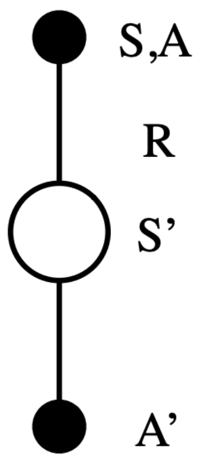
The SARSA, Q-Learning, and Expected SARSA algorithms return the $Q(S, A)$ values for every possible state–action pair. Once we obtain $Q(S, A)$, how is the policy defined? I assume it's obtained using the greedy choice: $\pi(S) = \text{argmax}_a \, Q(S, a)$, but I'd like to be sure.

Why is it not necessary to use importance sampling in Q-Learning?

Control: Q-learning vs. SARSA

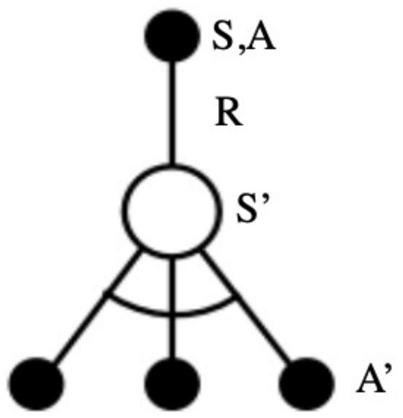
SARSA

Take action A, observe R, S'
Choose A' from S' using policy derived from Q (e.g., ε-greedy)
$Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma Q(S',A') - Q(S,A)\right]$
$S \leftarrow S'$; $A \leftarrow A'$;



Q-learning

Choose A from S using policy derived from Q (e.g., ε-greedy)
Take action A, observe R, S'
$Q(S,A) \leftarrow Q(S,A) + \alpha\left[R + \gamma \max_a Q(S',a) - Q(S,A)\right]$
$S \leftarrow S'$



Question #05

- **Importance sampling** is used to correct for the difference between the **behavior policy** b(a|s) (generates the data), and the **target policy** π(a|s) which we want to evaluate/improve.
- When data are collected under one policy but used to evaluate another, we normally need to weight updates by the ratio: π(a|s)/b(a|s)
- In Q-Learning, even though experience may come from a behavior policy b (for example, ε-greedy exploration), the update rule does not depend on b.

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]$$

- Notice that the next state update uses the **maximum over actions** which corresponds to following the **optimal policy**, not the behavior policy.
- Therefore, Q-Learning directly estimates the optimal action-value function, without needing to correct for how the data were collected.

Question #06

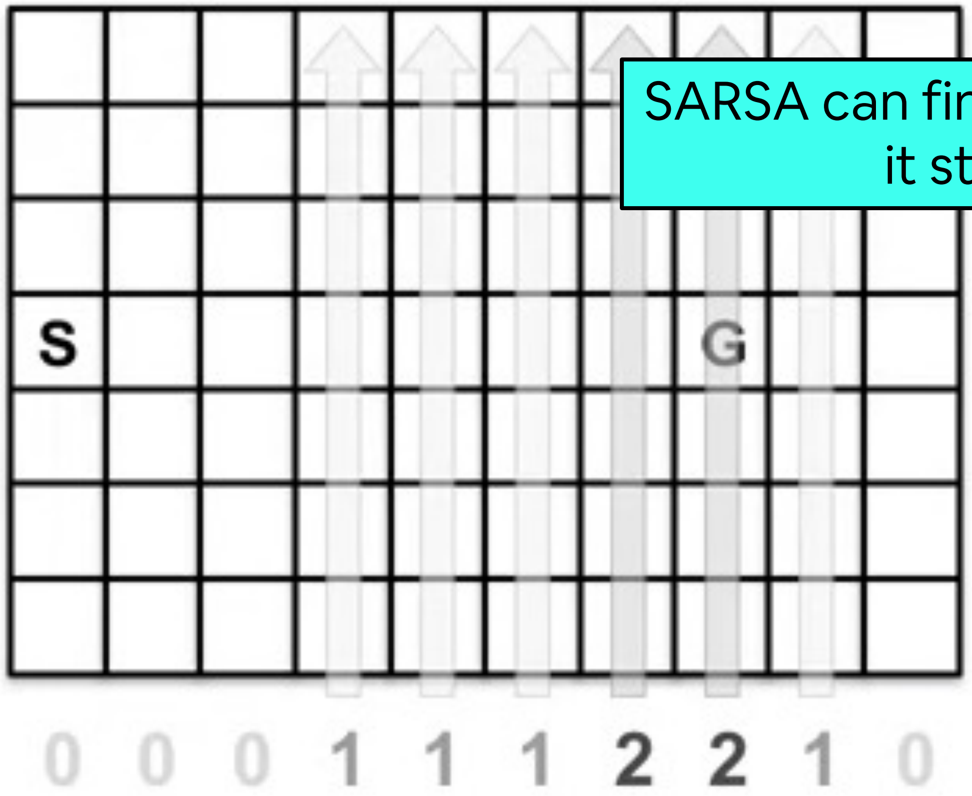
The question concerns why we introduced the concept of *off-policy*. I would like to understand whether the motivation is solely to decouple the management of the exploration–exploitation problem.

In fact, if I understood correctly, another motivation related to this idea was to obtain strategies/algorithms that lead to *optimal solutions* (fixing the shortcomings of on-policy Monte Carlo algorithms), which only happened in certain circumstances for *exploring starts*, or didn't happen at all for *on-policy Monte Carlo ε-greedy*.

However, we have only seen algorithms for the *control problem* in the context of *TD learning*. In this setting, it seems that *Q-learning* performs optimally in the example (the *cliff gridworld*) that we studied, but we did not discuss a general condition for optimality. In that same example, the other off-policy algorithm we saw was *Expected SARSA*, which did **not** return an optimal solution but one equivalent to *SARSA*.

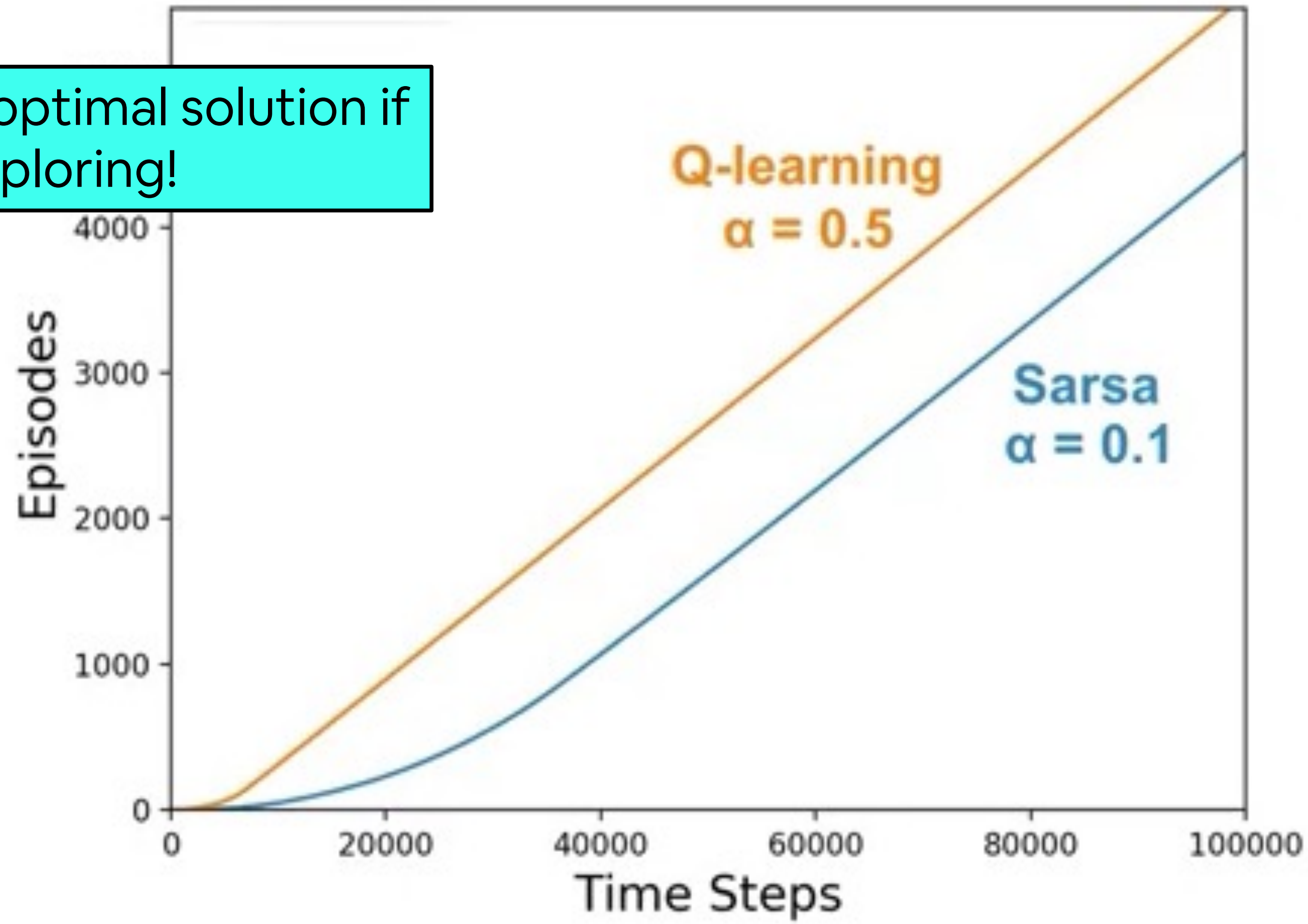
So, to recap, my question is:
Is the introduction of off-policy algorithms justified *solely* by the need to decouple the exploration–exploitation problem, or is the ultimate goal also to find strategies that always converge to an optimal policy? If the latter is true, then I don't understand why *Expected SARSA*, being off-policy, does not return an optimal policy. I would also like to clarify whether *Q-learning* is *always* optimal, or if that property only applied to the specific example shown in class.

Control: SARSA vs Q-learning – 'Windy Grid World' Example #01



$\epsilon = 0.1$

SARSA can find the optimal solution if it stops exploring!



A. White, M. White 'Sample-based Learning Methods'

Question #07

-**Why in importance sampling** W is multiplicated by itself?

Prediction: Off-policy learning

- In RL we have to deal with sequences!
- We use returns generated from b to evaluate π, we cannot directly compute v_π as the average of the returns seen!
- We have to correct each return thanks to importance sampling
- Given a starting state $S_t$the probability of the subsequent state-action trajectory under policy π is

$$
\begin{aligned}
&\Pr\{A_t, S_{t+1}, A_{t+1}, \ldots, S_T \mid S_t, A_{t:T-1} \sim \pi\} \\
&\quad = \pi(A_t|S_t)p(S_{t+1}|S_t, A_t)\pi(A_{t+1}|S_{t+1}) \cdots p(S_T|S_{T-1}, A_{T-1}) \\
&\quad = \prod_{k=t}^{T-1} \pi(A_k|S_k)p(S_{k+1}|S_k, A_k),
\end{aligned}
$$

Prediction: Off-policy learning

- In RL we have to deal with sequences!
- We use returns generated from b to evaluate π, we cannot directly compute v_π as the average of the returns seen!
- We have to correct each return thanks to importance sampling
- Given a starting state $S_t$the probability of the subsequent state-action trajectory under policy π is

$$
\begin{aligned}
&\Pr\{A_t, S_{t+1}, A_{t+1}, \ldots, S_T \mid S_t, A_{t:T-1} \sim \pi\} \\
&\quad = \pi(A_t|S_t)p(S_{t+1}|S_t, A_t)\pi(A_{t+1}|S_{t+1}) \cdots p(S_T|S_{T-1}, A_{T-1}) \\
&\quad = \prod_{k=t}^{T-1} \pi(A_k|S_k)p(S_{k+1}|S_k, A_k),
\end{aligned}
$$

Probability of a trajectory in an environment under policy π

Prediction: Off-policy learning

- The relative probability of the trajectory under the target and behaviour policy (the importance-sampling ratio) is

$$\rho_{t:T-1} \doteq \frac{\prod_{k=t}^{T-1} \pi(A_k|S_k)p(S_{k+1}|S_k, A_k)}{\prod_{k=t}^{T-1} b(A_k|S_k)p(S_{k+1}|S_k, A_k)} =$$

Probability of a trajectory in an environment under policy π

Probability of a trajectory in an environment under policy b

- We can now exploit returns obtain with b to estimate v_π with MC

$$\mathbb{E}[\rho_{t:T-1} G_t \mid S_t = s] = v_\pi(s)$$

- We can consider importance-sampling ratios as ‘weights’

Prediction: Off-policy learning

- The relative probability of the trajectory under the target and behaviour policy (the importance-sampling ratio) is:

$$\rho_{t:T-1} \doteq \frac{\prod_{k=t}^{T-1} \pi(A_k|S_k)\cancel{p(S_{k+1}|S_k, A_k)}}{\prod_{k=t}^{T-1} b(A_k|S_k)\cancel{p(S_{k+1}|S_k, A_k)}} = \prod_{k=t}^{T-1} \frac{\text{Probability of a trajectory in an environment under policy } \pi}{\text{Probability of a trajectory in an environment under policy } b}$$

- We can now exploit returns obtain with b to estimate v_π with MC

$$\mathbb{E}[\rho_{t:T-1} G_t \mid S_t = s] = v_\pi(s)$$

- We can consider importance-sampling ratios as 'weights'

Prediction: Off-policy learning

- The relative probability of the trajectory under the target and behaviour policy (the importance-sampling ratio) is:

$$\rho_{t:T-1} \doteq \frac{\prod_{k=t}^{T-1} \pi(A_k|S_k)p(S_{k+1}|S_k, A_k)}{\prod_{k=t}^{T-1} b(A_k|S_k)p(S_{k+1}|S_k, A_k)} = \prod_{k=t}^{T-1} \frac{\pi(A_k|S_k)}{b(A_k|S_k)}$$

- We can now exploit returns obtain with b to estimate v_π with MC

$$\mathbb{E}[\rho_{t:T-1}G_t \mid S_t = s] = v_\pi(s)$$

- We can consider importance-sampling ratios as ‘weights’

Monte Carlo off-policy Prediction

Input: a policy π to be evaluated

Algorithm 1 Off-policy Monte Carlo prediction algorithm for estimating $V \approx v_\pi$

Initialize:
    $V(s) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$
    $C(s) = 0$ (accumulating weights), for all $s \in \mathcal{S}$
    π as the target policy and b as the behavior policy (where $b(a|s) > 0$ if $\pi(a|s) > 0$)
Loop forever (for each episode):
    Generate an episode using b: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$
    $G \leftarrow 0$
    $W \leftarrow 1$
    For $t = T-1$ down to 0:
        $W \leftarrow W \cdot \frac{\pi(A_t|S_t)}{b(A_t|S_t)}$
        If $W = 0$ then:
            Break
        $G \leftarrow \gamma G + R_{t+1}$
        $C(S_t) \leftarrow C(S_t) + W$
        $V(S_t) \leftarrow V(S_t) + \frac{W}{C(S_t)}(G - V(S_t))$

Monte Carlo off-policy Prediction

Differences w.r.t. on-policy

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, **arbitrarily, for all** $s \in S$

$Returns(s) \leftarrow$ **an empty list, for all** $s \in S$

Loop forever (for each episode):

Generate an episode following $\boxed{b}$: $S_0, A_0, R_1, S_1, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$ $\boxed{W \leftarrow 1}$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$

$G \leftarrow \gamma \boxed{W} G + R_{t+1}$

Append G **to** $Returns(S_t)$

$V(S_t) \leftarrow$ **average**$(Returns(S_t))$

$$\boxed{W \leftarrow W \frac{\pi(A_t \mid S_t)}{b(A_t \mid S_t)}}$$

Monte Carlo off-policy Prediction

Differences w.r.t. on-policy

Input: a policy π to be evaluated

Initialize:

Initialization

$V(s) \in \mathbb{R}$, **arbitrarily, for all** $s \in S$

$Returns(s) \leftarrow$ **an empty list, for all** $s \in S$

Loop forever (for each episode):

Generate an episode following $\boxed{b}$: $S_0, A_0, R_1, S_1, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$ $\boxed{W \leftarrow 1}$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$

$G \leftarrow \gamma \boxed{W} G + R_{t+1}$

Append G **to** $Returns(S_t)$

$V(S_t) \leftarrow$ **average**$(Returns(S_t))$

$$\boxed{W \leftarrow W \frac{\pi(A_t \mid S_t)}{b(A_t \mid S_t)}}$$

Monte Carlo off-policy Prediction

Differences w.r.t. on-policy

We are using a different policy for data generation

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in S$

$Returns(s) \leftarrow$ an empty list, for all $s \in S$

Loop forever (for each episode):

Generate an episode following $\boxed{b}$: $S_0, A_0, R_1, S_1, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$ $\boxed{W \leftarrow 1}$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$

$G \leftarrow \gamma \boxed{W} G + R_{t+1}$

Append G to $Returns(S_t)$

$V(S_t) \leftarrow$ average$(Returns(S_t))$

$$\boxed{W \leftarrow W \frac{\pi(A_t \mid S_t)}{b(A_t \mid S_t)}}$$

Monte Carlo off-policy Prediction

Differences w.r.t. on-policy

We are correcting for the weights given by the Importance Sampling

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, **arbitrarily, for all** $s \in S$

$Returns(s) \leftarrow$ **an empty list, for all** $s \in S$

Loop forever (for each episode):

Generate an episode following $\boxed{b}$: $S_0, A_0, R_1, S_1 \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$ $\boxed{W \leftarrow 1}$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$

$G \leftarrow \gamma \boxed{W} G + R_{t+1}$

Append G **to** $Returns(S_t)$

$V(S_t) \leftarrow$ **average**$(Returns(S_t))$

$$\boxed{W \leftarrow W \frac{\pi(A_t \mid S_t)}{b(A_t \mid S_t)}}$$

Monte Carlo off-policy Prediction

Differences w.r.t. on-policy

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, **arbitrarily, for all** $s \in S$

$Returns(s) \leftarrow$ **an empty list, for all** $s \in S$

Loop forever (for each episode):

Generate an episode following $\boxed{b}$: $S_0, A_0, R_1, S_1, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$ $\boxed{W \leftarrow 1}$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$

$G \leftarrow \gamma \boxed{W} G + R_{t+1}$

Append G **to** $Returns(S_t)$

$V(S_t) \leftarrow$ **average**$(Returns(S_t))$

$$\boxed{W \leftarrow W \frac{\pi(A_t \mid S_t)}{b(A_t \mid S_t)}}$$

We are correcting for the weights given by the Importance Sampling

We are computing the 'weights' incrementally, backwards for efficiency!

$$\rho_{t:T-1} \doteq \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{b(A_k \mid S_k)}$$

$$= \rho_t \rho_{t+1} \rho_{t+2} \cdots \rho_{T-2} \rho_{T-1}$$

$$W_1 \leftarrow \rho_{T-1}$$

$$W_2 \leftarrow \rho_{T-1}\rho_{T-2}$$

$$W_3 \leftarrow \rho_{T-1}\rho_{T-2}\rho_{T-3}$$

Monte Carlo off-policy Prediction

Differences w.r.t. on-policy

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, **arbitrarily, for all** $s \in S$

$Returns(s) \leftarrow$ **an empty list, for all** $s \in S$

Loop forever (for each episode):

Generate an episode following $\boxed{b}$: $S_0, A_0, R_1, S_1 \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$ $\boxed{W \leftarrow 1}$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$; $\boxed{\textbf{while } W \neq 0}$

$G \leftarrow \gamma \boxed{W} G + R_{t+1}$

Append G **to** $Returns(S_t)$

$V(S_t) \leftarrow$ **average**$(Returns(S_t))$

$$\boxed{W \leftarrow W \frac{\pi(A_t \mid S_t)}{b(A_t \mid S_t)}}$$

Additional check to improve efficiency

Last Question

I have a doubt about the notation used in Lecture 6, specifically in the Policy Improvement Theorem.

From what I understand, $q_\pi(s, \pi'(s))$ represents the expected return when starting in state s, taking action $\pi'(s)$, and then following policy π.

If that's the case, I don't quite understand why the expected value is written with the subscript π'. Does this subscript simply indicate that the *first* action is taken according to π'?

In Lecture 4, the subscript denoted the policy being followed, so I'm a bit confused here.

Last Question

I have a doubt about the notation used in Lecture 6, specifically in the Policy Improvement Theorem.

From what I understand, $q_\pi(s, \pi'(s))$ represents the expected return when starting in state s, taking action $\pi'(s)$, and then following policy π.

If that's the case, I don't quite understand why the expected value is written with the subscript π'. Does this subscript simply indicate that the *first* action is taken according to π'?

In Lecture 4, the subscript denoted the policy being followed, so I'm a bit confused here.

Control: Policy Improvement Theorem

- Let's consider a deterministic policy π
- We can improve the policy by acting greedily: $\pi'(s) = \underset{a \in \mathcal{A}}{\operatorname{argmax}}\ q_\pi(s, a)$

(that is, we pick the one action that provides the best value and then we follow π - π' is the greedy policy)

- This improves for one step: $q_\pi(s, \pi'(s)) = \max_{a \in \mathcal{A}} q_\pi(s, a) \geq q_\pi(s, \pi(s)) = v_\pi(s)$
- And improves the whole value function

$$
\begin{aligned}
v_\pi(s) &\leq \boxed{q_\pi(s, \pi'(s)) = \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s\right]} \\
&\leq \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma q_\pi(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s\right] \\
&\leq \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 q_\pi(S_{t+2}, \pi'(S_{t+2})) \mid S_t = s\right] \\
&\leq \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma R_{t+2} + \ldots \mid S_t = s\right] = v_{\pi'}(s)
\end{aligned}
$$

After the next step, everything is equal (we are following π after the first step)

Control: Policy Improvement Theorem

- Let's consider a deterministic policy π
- We can improve the policy by acting greedly: $\pi'(s) = \underset{a \in \mathcal{A}}{\operatorname{argmax}}\, q_\pi(s, a)$

(that is, we pick the one action that provides the best value and then we follow π - π' is the greedy policy)

- This improves for one step: $q_\pi(s, \pi'(s)) = \max_{a \in \mathcal{A}} q_\pi(s, a) \geq q_\pi(s, \pi(s)) = v_\pi(s)$
- And improves the whole value function

$$
\begin{aligned}
v_\pi(s) \leq q_\pi(s, \pi'(s)) &= \boxed{\mathbb{E}_{\pi'}\left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s\right]} \\
&\leq \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma q_\pi(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s\right] \\
&\leq \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 q_\pi(S_{t+2}, \pi'(S_{t+2})) \mid S_t = s\right] \\
&\leq \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma R_{t+2} + \ldots \mid S_t = s\right] = v_{\pi'}(s)
\end{aligned}
$$

I'm applying this again...

$$
\begin{aligned}
v_\pi(s) &\le q_\pi(s, \pi'(s)) \\
&= \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = \pi'(s)] && \text{(by (4.6))} \\
&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] \\
&\le \mathbb{E}_{\pi'}[R_{t+1} + \gamma q_\pi(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s] && \text{(by (4.7))} \\
&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma \mathbb{E}_{\pi'}[R_{t+2} + \gamma v_\pi(S_{t+2}) | S_{t+1}, A_{t+1} = \pi'(S_{t+1})] \mid S_t = s] \\
&= \mathbb{E}_{\pi'}\big[R_{t+1} + \gamma R_{t+2} + \gamma^2 v_\pi(S_{t+2}) \bigm| S_t = s\big] \\
&\le \mathbb{E}_{\pi'}\big[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 v_\pi(S_{t+3}) \bigm| S_t = s\big] \\
&\;\;\vdots \\
&\le \mathbb{E}_{\pi'}\big[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots \bigm| S_t = s\big] \\
&= v_{\pi'}(s).
\end{aligned}
$$

Thank you!
Questions?
Lecture #11
Rewind: The Story So Far

Gian Antonio Susto